

Software Architecture and Design Specification

Project: Bank Management System

Status: Draft

Team Members

Name	SRN
Kruthik Reddy.S	PES2UG24CS241
Kotipalli Devi Sai Charan	PES2UG24CS231
Kiran BG	PES2UG24CS227

Course Coordinator: Sheela Devi

Revision History

Version	Date	Author	Change Summary	Approval
1.0	06-09-2026	Student Team	Initial SAD prepared for Bank Management System based on the supplied SRS.	Pending

Approvals

Role	Name	Signature / Email	Date
Course Coordinator	Sheela Devi		

1. Introduction

1.1 Purpose

This Software Architecture and Design Specification (SAD) describes the architecture, components, interfaces, security design, and detailed design of the Bank Management System. It is derived from the supplied SRS and is intended to guide implementation, testing, maintenance, and evaluation.

1.2 Scope

The system is a standalone C/C++ console-based application supporting customer account creation and authentication, deposits, withdrawals, balance inquiry, transaction history, administrative account management, and basic report generation. Data is persisted in local binary/text files. Online banking, mobile applications, payment-gateway integration, real-time interbank transfers, and multi-branch/distributed processing are outside the current scope.

1.3 Audience

Developers, course instructor/evaluator, QA reviewers, security reviewers, maintenance teams, and end users (Customers and Bank Admin/Staff).

1.4 Definitions

SRS: Software Requirements Specification; SAD: Software Architecture and Design Specification; UI: User Interface; CLI: Command-Line Interface; I/O: Input/Output; PIN: Personal Identification Number; CRUD: Create, Read, Update, Delete; RBAC: Role-Based Access Control; EOD: End of Day.

2. Document Overview

2.1 How to Use This Document

The document describes system architecture, component responsibilities, design interfaces, security controls, error handling, user-interface considerations, traceability, and future work.

2.2 Related Documents

Primary related document: Software Requirements Specification (SRS), Project: Bank Management System, Version 1.0, dated 06-09-2026.

3. Architecture

3.1 Goals & Constraints

- Separate the console interface, authentication, account management, transactions, reporting, and file storage into logical modules.
- Protect customer PINs, balances, and administrative functions from unauthorized access.
- Prevent invalid transactions and negative account balances.

- Maintain transaction and audit history in persistent local files.
- Operate without network connectivity or a remote database.
- Support Windows/Linux desktop environments with a C/C++ compiler.

3.2 Stakeholders & Concerns

Stakeholder	Primary Concerns
Customer	Secure authentication, accurate balances, reliable deposits/withdrawals, clear prompts.
Admin / Bank Staff	Account lifecycle management, account search, transaction review, reporting, security.
Course Instructor / Evaluator	Requirement coverage, correctness, architecture clarity, testability.
QA Reviewers	Acceptance criteria, invalid-input handling, reliability, security, traceability.
Developers / Maintenance	Modularity, readable interfaces, maintainable file and transaction handling.

3.3 Component (UML) Diagram

The system is organized into logical components: Console UI, Authentication Module, Account Module, Transaction Module, Admin Module, Report Module, and File Storage Layer.

- Console UI -> Authentication Module: login and role verification.
- Console UI -> Account Module: account creation, profile updates, and balance inquiry.
- Console UI -> Transaction Module: deposit, withdrawal, and transaction recording.
- Console UI -> Admin Module: customer-account administration and account search.
- Console UI -> Report Module: account and transaction summary reports.
- Authentication, Account, Transaction, Admin, and Report modules interact with the File Storage Layer for persistent data.

3.4 Component Descriptions

Component	Responsibilities
-----------	------------------

Console UI	Displays menus/prompts, collects input, and displays results/errors.
Authentication Module	Customer login using account number/PIN, Admin authentication, and attempt lockout.
Account Module	Create accounts, generate/validate unique account numbers, update profiles, and retrieve balances.
Transaction Module	Process deposits/withdrawals, validate amounts and balances, and record transactions.
Admin Module	View, edit, freeze, close/delete, and search customer accounts.
Report Module	Generate account summary and per-account transaction reports.
File Storage Layer	Persist accounts, transactions, and reports using local files.

3.5 Chosen Architecture Pattern and Rationale

A layered architecture is selected. The Console/UI layer handles interaction, the Application/Service layer coordinates use cases, the Domain/Core layer contains account and transaction rules, and the Persistence layer handles local file I/O. This matches the standalone C/C++ and file-driven nature of the system. A distributed or microservices architecture is outside the current scope because the SRS excludes networked and distributed processing.

3.6 Technology Stack & Data Stores

Area	Technology / Design
Implementation	C / C++
Interface	Command-line / menu-driven console UI
Compiler / Environment	GCC/G++ or equivalent on Windows/Linux
Persistence	Local binary/text files using fstream / FILE I/O

Data Models	C/C++ structures such as Account and Transaction
Reports	Plain text/CSV output
Network	None in current version

3.7 Risks & Mitigations

Risk	Mitigation
File corruption during writes	Controlled file-update operations and integrity checks; forced-exit testing.
Invalid numeric input	Validate input before processing and leave balances unchanged on failure.
Unauthorized Admin access	Require separate Admin authentication before Admin-only menus.
PIN exposure	Suppress/mask PIN entry where supported and never display PINs during re-entry.
Negative balance	Validate withdrawal against current balance before updating.
Account duplication	Auto-generate or validate unique account numbers.

3.8 Traceability to Requirements

Requirement	Architectural Component
BMS-F-002 Login & PIN authentication	Authentication Module
BMS-F-010 Deposit	Transaction Module
BMS-F-011 Withdrawal	Transaction Module
BMS-F-020 Balance inquiry	Account Module
BMS-F-030 View all customer accounts	Admin Module
BMS-F-033 Close/delete account	Admin Module

BMS-F-040 Account summary report	Report Module
BMS-NF-001 Response time	Console UI / Core Modules
BMS-SR-002 PIN attempt lockout	Authentication Module

3.9 Security Architecture

- Authentication: customer access requires account number and PIN; Admin functions require the separate Admin password.
- Authorization: customer menus are separated from Admin-only operations.
- PIN protection: PIN entry should be masked/suppressed where supported; PINs should not be displayed during re-entry.
- Login protection: after three consecutive incorrect PIN attempts within a session, further attempts are locked out.
- Transaction protection: deposits must be positive; withdrawals must be positive and cannot exceed the available balance.
- Auditability: deposits, withdrawals, and Admin account-management actions are recorded in an append-only, timestamped audit/transaction log.

4. Design

4.1 Design Overview

The system uses modular C/C++ functions and structures organized around banking responsibilities. The Console UI invokes the appropriate module, business rules are applied before state changes, and the persistence layer synchronizes account and transaction records with local files.

4.2 UML Sequence Diagrams

At least two sequence diagrams are required. The following flows are specific to the supplied SRS.

4.2.1 Customer Login and Deposit

- Customer -> Console UI: Enter account number and PIN.
- Console UI -> Authentication Module: Validate credentials.
- Authentication Module -> File Storage: Read account/authentication record.

- File Storage -> Authentication Module: Return matching record.
- Authentication Module -> Console UI: Authentication result.
- Customer -> Console UI: Enter deposit amount.
- Console UI -> Transaction Module: Validate deposit.
- Transaction Module -> Account/File Storage: Update balance.
- Transaction Module -> Transaction Log: Append timestamped deposit.
- Console UI -> Customer: Display success and updated balance.

4.2.2 Admin Freezes an Account

- Admin -> Console UI: Enter Admin password.
- Console UI -> Authentication Module: Validate Admin credentials.
- Authentication Module -> Console UI: Authorization result.
- Admin -> Console UI: Select customer account and Freeze.
- Console UI -> Admin Module: Request account freeze.
- Admin Module -> Account/File Storage: Set account status to frozen.
- Account/File Storage -> Admin Module: Confirm update.
- Console UI -> Admin: Display confirmation.

4.3 Interface Design

Because the SRS specifies a local console application with no client-server communication, interfaces are defined as module/function contracts rather than HTTP APIs.

Interface	Inputs	Outputs / Errors
Authentication Module	Account number, PIN	Authenticated/Rejected; lockout after 3 incorrect attempts.
Transaction Module	Account ID, transaction type, amount	Updated balance or validation/insufficient-balance error.
Account Module	Customer details, account ID	Account record, updated profile, or validation error.

Admin Module	Admin credentials, account ID, action	Account-management result or authorization error.
Report Module	Report type, optional account/date range	Text/CSV report file or report-generation error.

4.4 Error Handling, Logging & Monitoring

- Reject zero, negative, and non-numeric transaction amounts.
- Reject withdrawals that exceed the current balance.
- Display clear error prompts and do not modify account data after failed validation.
- Handle invalid menu choices without terminating the application.
- Maintain timestamped transaction/audit records for successful deposits, withdrawals, and Admin account-management actions.
- Avoid exposing sensitive PIN information in console output or logs.
- Test data-file integrity after forced termination during a write operation.

4.5 UX Design

The primary interface is a text-based, menu-driven console. Customer and Admin menus are separated. Prompts should clearly identify required inputs, success messages, and validation errors. The SRS requires the console UI to be understandable to a first-time user without external documentation.

4.6 Open Issues & Next Steps

- Complete and verify the UML component and sequence diagrams.
- Implement and test all high-priority functional requirements.
- Complete the Requirements Traceability Matrix with final test status.
- Verify file-integrity behavior under abrupt program termination.
- Validate PIN masking/suppression across the target platform.
- Future enhancements may include stronger credential protection, richer reporting, and a graphical or networked version if the scope is expanded.

5. Appendices

5.1 Glossary

Term	Meaning
SRS	Software Requirements Specification
SAD	Software Architecture and Design Specification
PIN	Personal Identification Number
CRUD	Create, Read, Update, Delete
RBAC	Role-Based Access Control
CLI	Command-Line Interface
EOD	End of Day
RTM	Requirements Traceability Matrix

5.2 References

Primary reference: supplied Software Requirements Specification (SRS), Bank Management System, dated 06-09-2026. The SRS contains the requirements, acceptance criteria, UML use-case diagrams, and RTM used as the basis for this SAD.

5.3 Tools

C/C++ compiler such as GCC/G++, standard file I/O facilities, and UML diagramming tools such as PlantUML or draw.io for architecture and sequence diagrams.